

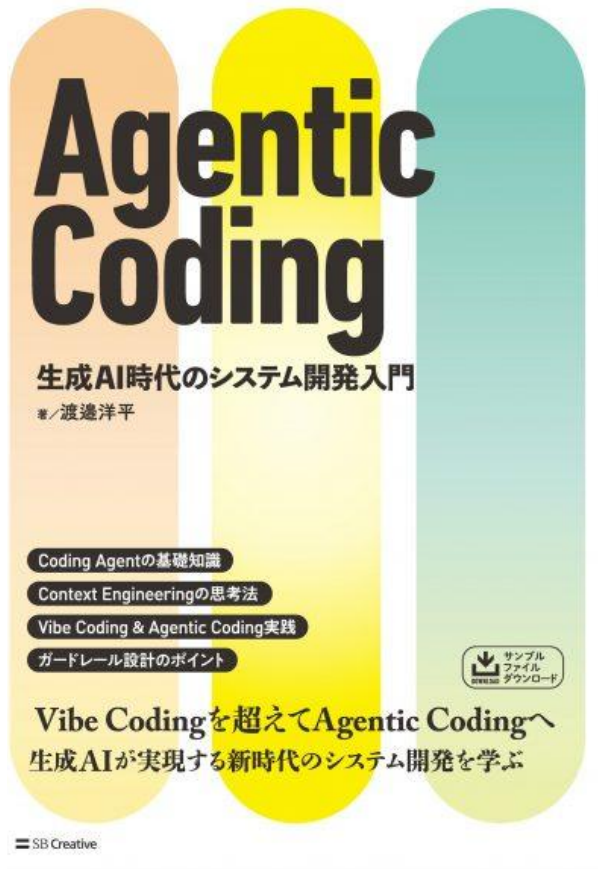
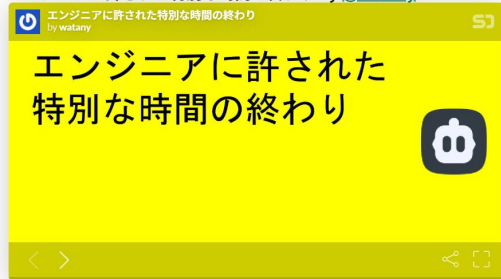
仕様駆動開発の 消費期限

About Me

渡邊 洋平(watany)

- 所属：NTTテクノクロス株式会社
- AWS Ambassadors(2024～)
- JAWS-UG東京(AWSコミュニティ) 運営
- 寄稿：CodeZine
 - 仕様駆動開発への期待と誤解

エンジニアに許された特別な時間の終わり by @watany



2026年5月2日(土)発売

<https://www.sbcr.jp/product/4815636593/>

うまい棒(納豆味)はからしの風味が美味しい

おかしいすき!

やおきん

 すべての商品

 うまい棒について

 知る・楽しむ

 お店一覧



うまい棒納豆味

納豆パウダーを使用し、本格的な味に仕上げましたうまい棒です。

内容量	6g
賞味期限	150日
JANコード	49480771
メーカー小売価格	フリープライス

からしがなくても納豆の味は成り立つ

|| 臭い消し

そもそも納豆にカラシを入れる大きな目的は、きつい臭いを打ち消すことだった。そのルーツは納豆が庶民の食べ物として定着した江戸時代まで遡る。

当時、貴重なたんぱく源だったが、納豆菌の発酵管理は難しく、主に気温の低い冬場に作られていた。ただ、冷蔵保存の手段が乏しく、再発酵によってアンモニア臭が発生することが大きな問題だった。その臭いを抑えるために用いられたのがカラシだ。安価で入手しやすく、庶民の間で広まっていった。

タレやカラシが添付された商品が売られるようになったのは1970年代前半だ。個食化の進展で、家長が混ぜた納豆を家族で分ける風習が廃れたことと、小袋への装填^{そうてん}技術が発達したからだという。また、冷蔵保存技術や物流網の発達によって、臭いの問題も軽減された。カラシの役割は風味付けに比重が移った。

「無くても成り立つもの」と
どう向き合うか

仕様駆動開発について

Specification(仕様書)について

> A specification is a kind of
(version controlled,
human-readable) super prompt.

—
仕様書とは、一種の(バージョン
管理され、人間が読める)スー
パープロンプトです。

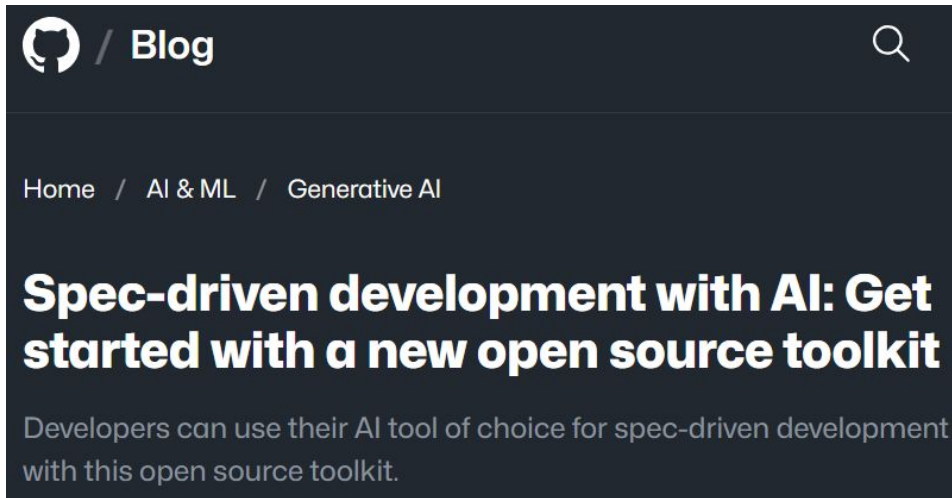


Specification(仕様書)について

> That's why we're rethinking specifications — not as static documents, but as living, executable artifacts that evolve with the project.

—

仕様書を静的な文書ではなく、プロジェクトと共に進化する生きた実行可能な成果物として捉え直しています。



仕様駆動開発 (Spec-Driven Development, SDD)

2025/7、AWSがIDE「Kiro」の発表と共に提唱

- 「実装前に仕様(書)を作る」というVibe Coding以後のレトロニム
 - レトロニム：新しい事物の誕生後、既存の事物を区別するため「後から」作られた言葉。例) 固定電話、オンプレミス
- 実用として以下の側面がある
 1. フレームワークとしてのAIワークフロー
 2. エージェント用のドキュメント管理(長期記憶)



Kiro
<https://kiro.dev/>

”仕様駆動開発”の定義が拡張されているケース



なぜ「7つ」なのか — 英語圏の潮流との違い

ここが、私の方法論の特徴です。

GitHub は公式の技術ブログ記事「Spec-driven development with AI: Get started with a new open source toolkit」で、**4つのプロセス (Specify / Plan / Tasks / Implement)** を示しました（この記事が、オープンソースのツールキット Spec Kit を紹介したものです）。私はこの4プロセスを土台に、**日本の組織特性に合わせて3つの工程を足し、7つに拡張しました。**

足したのは、次の3つです。

- **原則決定 (Constitution)** — 合意形成文化に應える。プロジェクトの存在意義・基本原則を最初に明文化し、全員で合意する
- **検証・受入 (Verify / Accept)** — 品質管理に應える。継続的な検証と、最終的な受入を工程として明示する
- **移行・運用 (Migration / Operation)** — 運用のフィードバックを次の仕様に戻す。開発と保守を分断しない

つまり — 英語圏は「速く作る」ための4工程。私ののは、**合意形成・契約・品質**といった日本の**開発現場の現実**に、仕様駆動開発を着地させるための7工程です。同じ潮流に立ちながら、向き合っている現場が違う。そこが違いです。

仕様駆動開発 (Spec-Driven Development, SDD)

2025/7、AWSがIDE「Kiro」の発表と共に提唱

- 「実装前に仕様(書)を作る」というVibe Coding以後のレトロニム
 - レトロニム：新しい事物の誕生後、既存の事物を区別するため「後から」作られた言葉。例) 固定電話、オンプレミス
- 実用として以下の側面がある
 1. フレームワークとしてのAIワークフロー
 2. エージェント用のドキュメント管理(長期記憶)



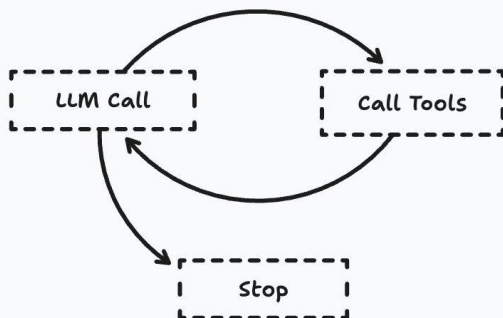
Kiro
<https://kiro.dev/>

1. フレームワークとしてのAIワークフロー

※AIワークフロー：エージェントにステップを伝える

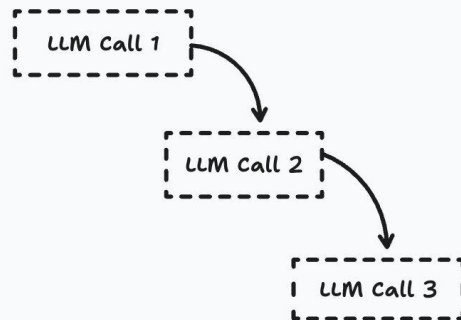
○ Agent

A loop where the agent decides when to stop.

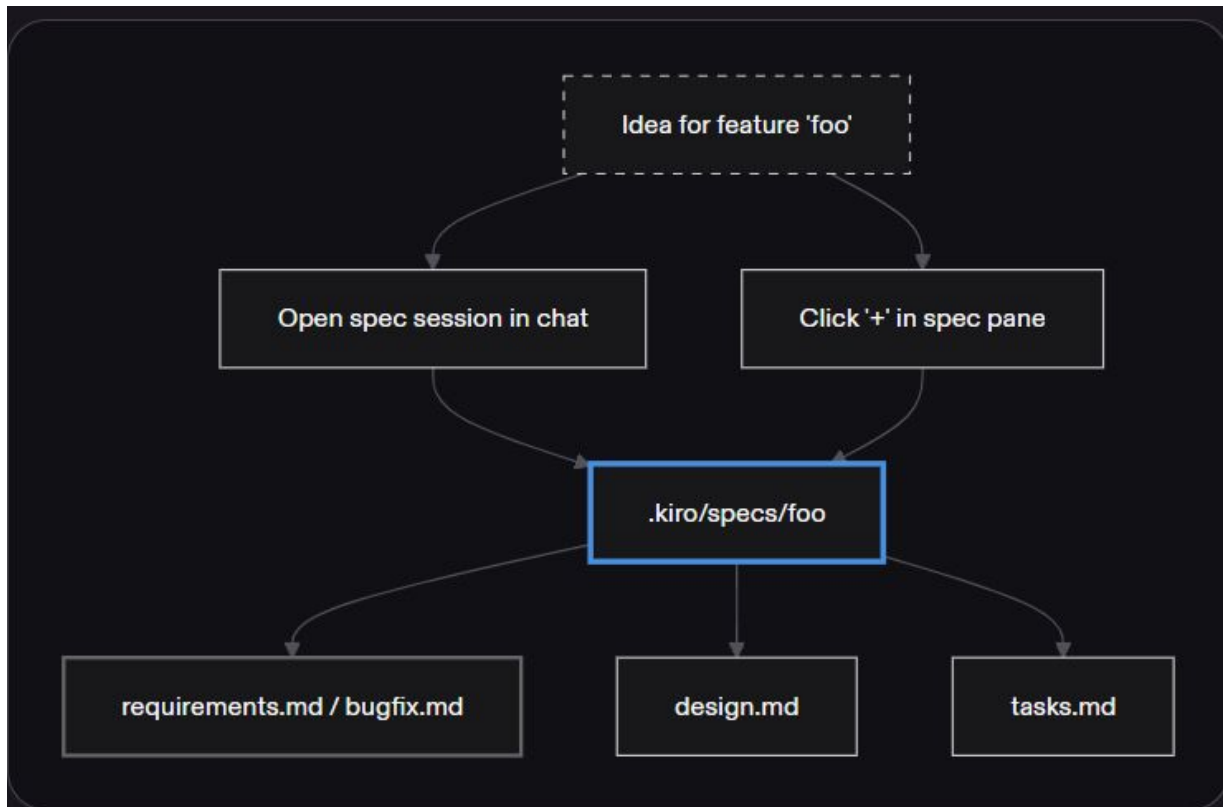


○ Workflow

Predetermined steps, written in code

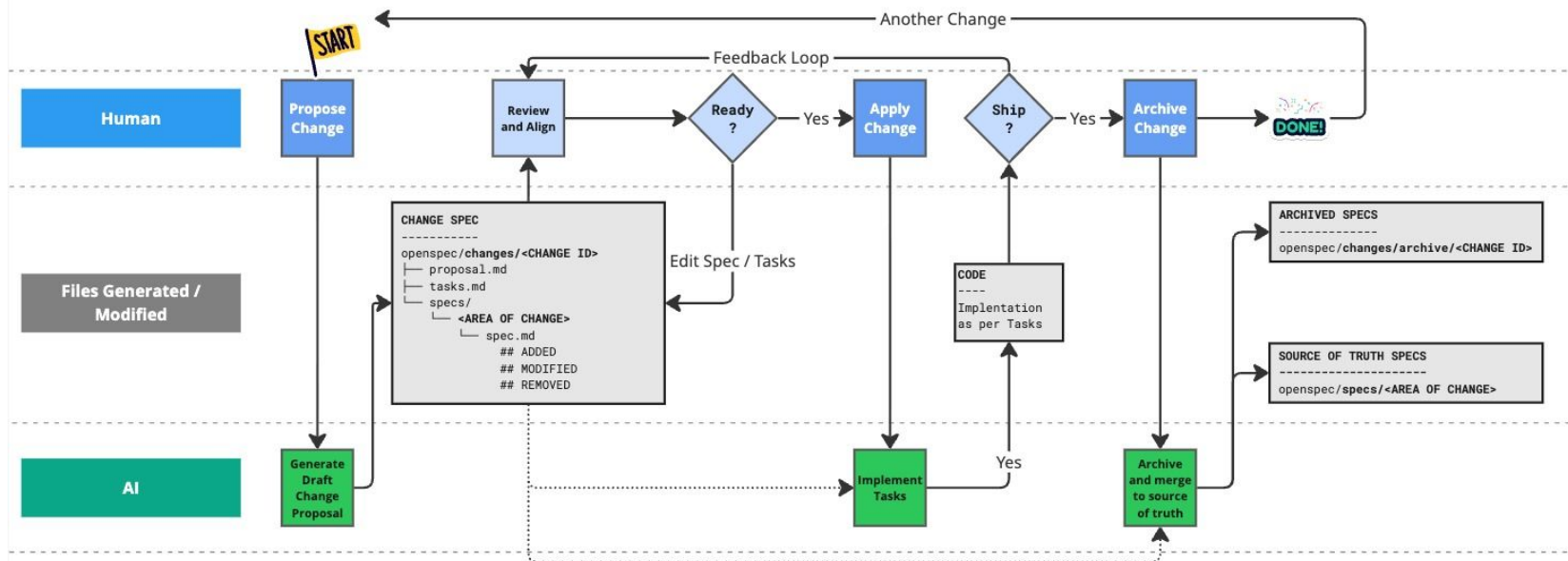


仕様駆動開発のAIワークフロー(Kiro)



仕様駆動開発のAIワークフロー(Openspec)

OpenSpec - Change Workflow



Hari Krishnan

🌐 harikrishnan.io

🌐 linkedin.com/in/harikrishnan83

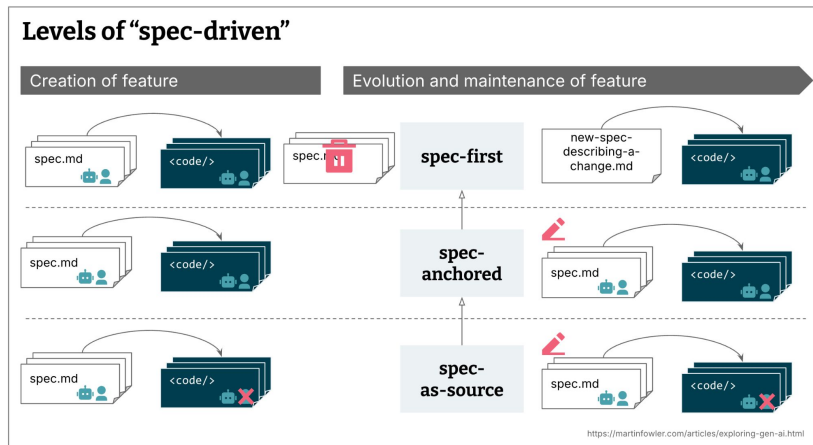
仕様駆動開発のAIワークフロー(共通点)

- 仕様駆動開発は、実態として「仕様→設計→タスク化」のワークフローを核にしている
 - Kiro: Requirements → Design → Tasks
 - Spec-Kit : Specify → Plan → Tasks
 - OpenSpec : Proposal → Specs → Design → Tasks

2. エージェント用のドキュメント管理(長期記憶)

spec実装のパターン

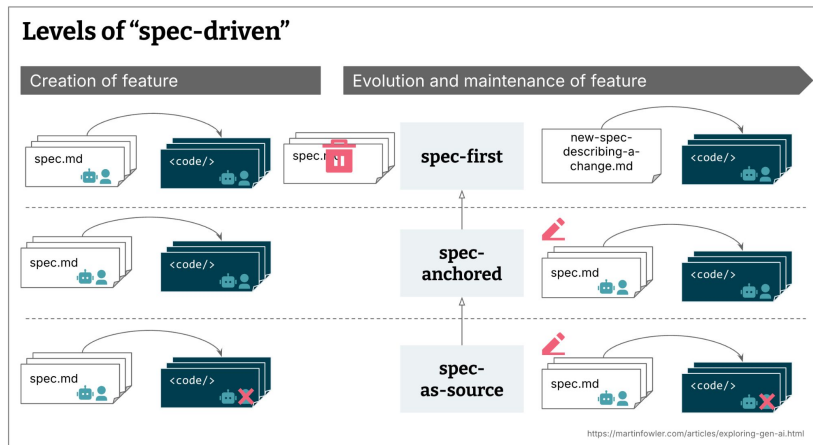
1. spec-first: まずspecを書いて開発。タスク完了後、specを破棄する
2. spec-anchored: タスク完了後もspecを保持。将来の変更・保守でもspecを更新し続ける
3. spec-as-source: specを原本としてコードは生成される。人は直接編集しない



※注：事実上、実用されているのは2種類

spec実装のパターン

1. spec-first: まずspecを書いて開発。タスク完了後、specを破棄する
2. spec-anchored: タスク完了後もspecを保持。将来の変更・保守でもspecを更新し続ける
3. ~~spec as source: specを原本としてコードは生成される。人は直接編集しない~~



仕様駆動開発について、理解が深まった

「いつの話をしてんの？」

”仕様駆動開発”が生まれた時期

- 前述の「仕様駆動開発のワークフロー」は2025/7～2025/9頃に検討されたもの
 - 本登壇(2026/8/6)の約一年前
- 2025/7～2025/9(一年前)に主流だったモデル
 - Claude Sonnet 4, Opus 4.1
 - GPT-4o、GPT-4.1、o3 → GPT-5
 - DeepSeek V3.1, Qwen3-Coder, gpt-oss

LLMモデルの転換点

”仕様駆動開発”(2025/9)以後の自走性能向上

—
> (私訳)2025年11月の GPT-5.2 と Opus 4.5 は、本当に転換点を表しているように感じます。

Simon Willison's Weblog

Sponsored by: Postman — Every API your agents depend on, mapped and monitored. [See what](#)

It genuinely feels to me like GPT-5.2 and Opus 4.5 in November represent an inflection point - one of those moments where the models get incrementally better in a way that tips across an invisible capability line where suddenly a whole bunch of much harder coding problems open up.

<https://x.com/karpathy/status/2026731645169185220>



Simon Willison

Djangoの共同開発者

Spec-firstとは”いつものPlanモード”に見える

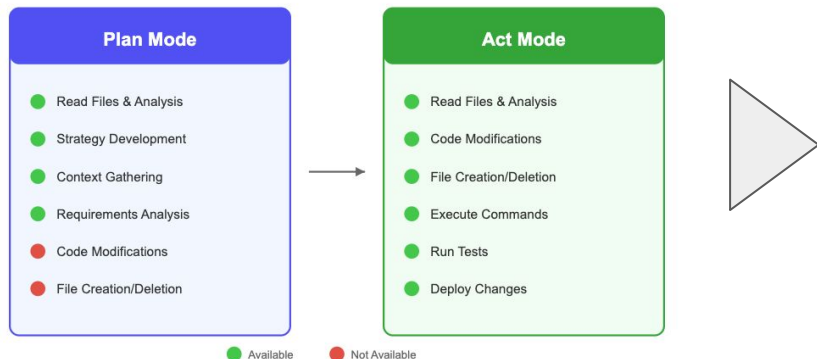
> また、仕様駆動開発のうち
Spec-firstのスタイルはPlanモード
に合流し、Coding Agentの多くの
実装に含まれる当たり前のものにな
りつつあります。



Spec-firstとは”いつものPlanモード”に見える

- spec-first : まずspecを書いて開発。タスク完了後specを破棄
- Plan Mode : 計画・設計から作成したタスクリストの通りに実装
 - 初期はReadOnly(Ask)だったが、spec-first化が主流

Plan vs Act Mode Capabilities



Planモード不要論

(和訳)

> 私はPlanモードを使いません。

> (Planモードが)Codexに追加された理由は「Claude脳」の利用者に向けた機能だからです。

> ただエージェントと普通に会話すればいいのです。



Peter Steinberger 🦄 🟦 🇺🇸
@steipete



I never use plan mode.

The main reason this was added to codex is for claude-pilled people who struggle with changing their habits.

just talk with your agent.

Peter Steinberger

OpenClawの開発者

<https://x.com/steipete/status/2039551079621566812>

考えて保存するだけならPlanやSpecは必要ない

Skills : mattpocock/grill-with-docs

- Agentが利用者に一問ずつ質問する
- 決まった用語を用語集 (CONTEXT.md) に記録
- 重要な設計判断を docs/adr/ に記録



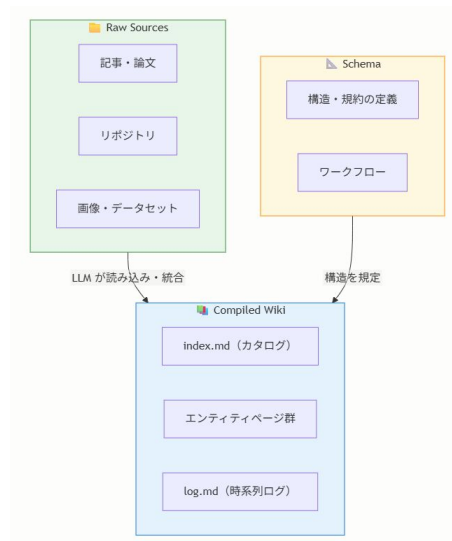
Matt Pocock

講座「Total TypeScript」の作者

エージェント用の記録方法はSpec以外にもある

LLM Wiki(LLM Knowledge Base) Open Knowledge Format(OKF)

- Andrej Karpathy提唱のナレッジベース管理手法



- Google提唱のナレッジ標準化フォーマット

```
sales/
├─ index.md
├─ datasets/
│   └─ index.md
│       └─ orders_db.md
├─ tables/
│   └─ index.md
│       └─ orders.md
│           └─ customers.md
└─ metrics/
    └─ index.md
        └─ weekly_active_users.md
```

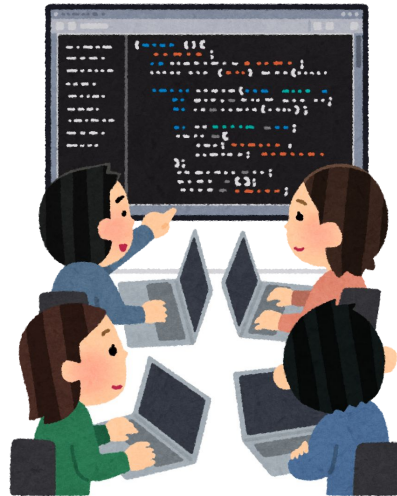
仕様駆動開発の消費期限

- 「仕様駆動開発のワークフロー」は2025/7～2025/9頃に検討されたもの
 - 本登壇(2026/8/6)の約一年前
- 実用として以下の側面がある
 - フレームワークとしてのAIワークフロー
 - 「事前に作成したタスクリストの実装」はエージェントハーネスに一般化されている
 - エージェント用のドキュメント管理(長期記憶)
 - 代替のプラクティスが発見され、唯一無二ではない

仕様駆動開発の 消費期限

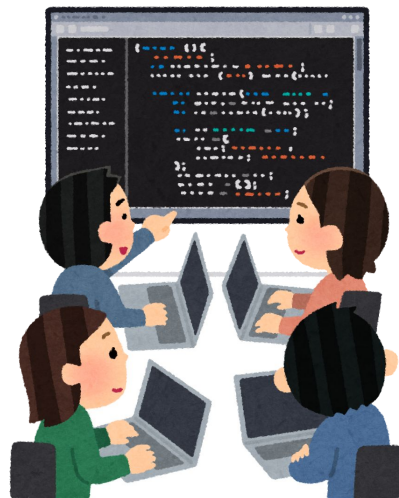
チームで開発すると別のものが見えてくる

ソロプレイ VS チームプレイ



実際にあった依頼(2026/04)

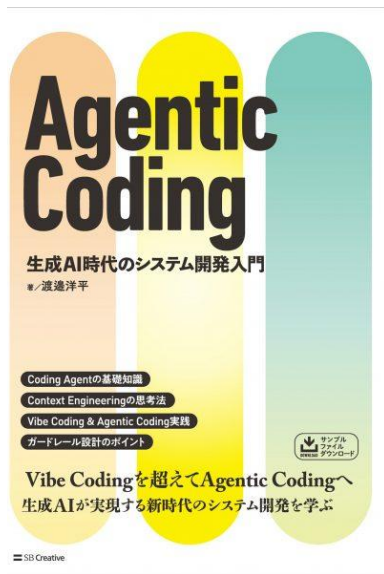
Agent(CLI)を使うが、
AI駆動開発の進め方を
チームで決めたいので
提案してほしい



仕様駆動開発を想定して
いるが、もっといい手法
があったらそれを採用し
たい

AI駆動開発の「守」

- AI駆動開発について「守破離」以前に守るべき型が無い状況
 - ”エンジニアたるもの月3000円～30000円を負担し、余暇で慣れる”という特殊なムードが前提に置されている
 - 自著では可能な限り「型」のベースを体系化したが、チーム全員に定着させるには難しさがある



AI駆動開発と「守破離」

- 守破離の3ステップ
 - 型を守る
 - 型を破る
 - 型を離れる
- では、AI駆動開発の「型」とは何を指しているか？

AI駆動開発のフレームワーク：AI-DLC

AI-DLCの理論：AI-DLC Whitepaper

AI-DLC の仕組み

AI-DLC の本質は、新しいメンタルモデルを通じて AI にワークフローを開始させ、指示することです：



このパターンでは、AI が計画を作成し、文脈を理解するためにすり合わせのための質問をし、人間の検証を受けた後にのみソリューションを実装します。このパターンは、すべての SDLC 活動に対して迅速に繰り返され、すべての開発パスに統一されたビジョンとアプローチを提供します。

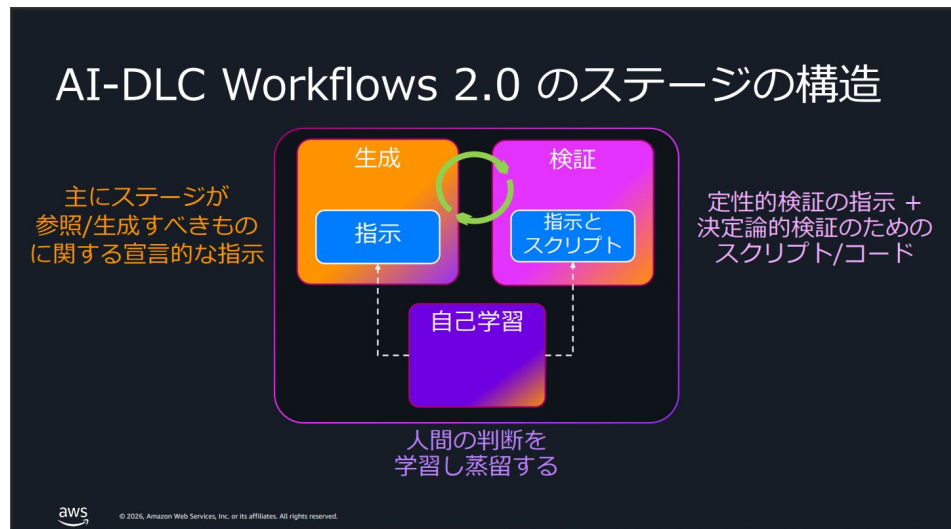
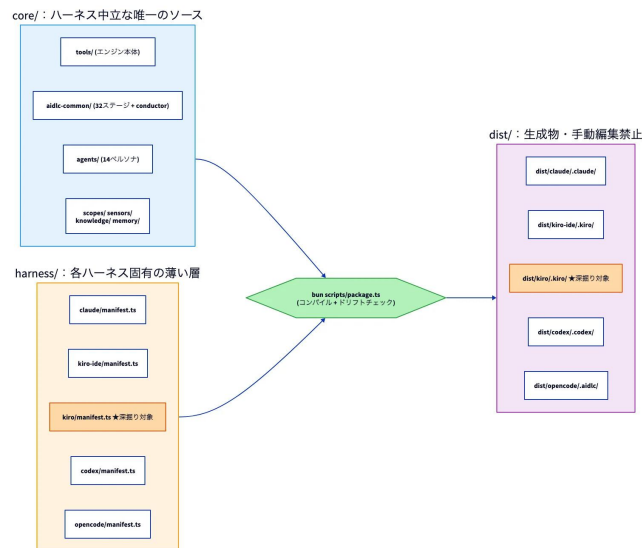
このメンタルモデルを核として、AI-DLC でのソフトウェア開発は3つシンプルなフェーズで行われます：

- **開始(Inception)**フェーズでは、AI がビジネス意図を詳細な要件、ストーリー、ユニットに変換します。これは「モブエラボレーション(Mob Elaboration (※))」を通じて行われ、チーム全体が AI の質問や提案を積極的に検証します。(※ 訳註: elaboration は「推敲」や「入念な準備」という意味を持つ英単語です)
- **構築(Construction)**フェーズでは、開始フェーズで検証されたコンテキストを使用して、AI が論理アーキテクチャ、ドメインモデル、コードによる実装、テストを「モブコンストラクション (Mob Construction)」を通じて提案します。ここでは、チームが技術的決定とアーキテクチャの選択についてリアルタイムで明確化していきます。
- **運用(Operation)**フェーズでは、AI が前のフェーズから蓄積されたコンテキストを適用して、チームの監督のもとで Infrastructure as Code とデプロイメントを管理します。

各フェーズは次のフェーズにより豊富なコンテキストを提供し、AI がより的確な提案を提供できるようにします。

AI駆動開発のフレームワーク：AI-DLC

AI-DLCの実践：AI-DLC Workflows

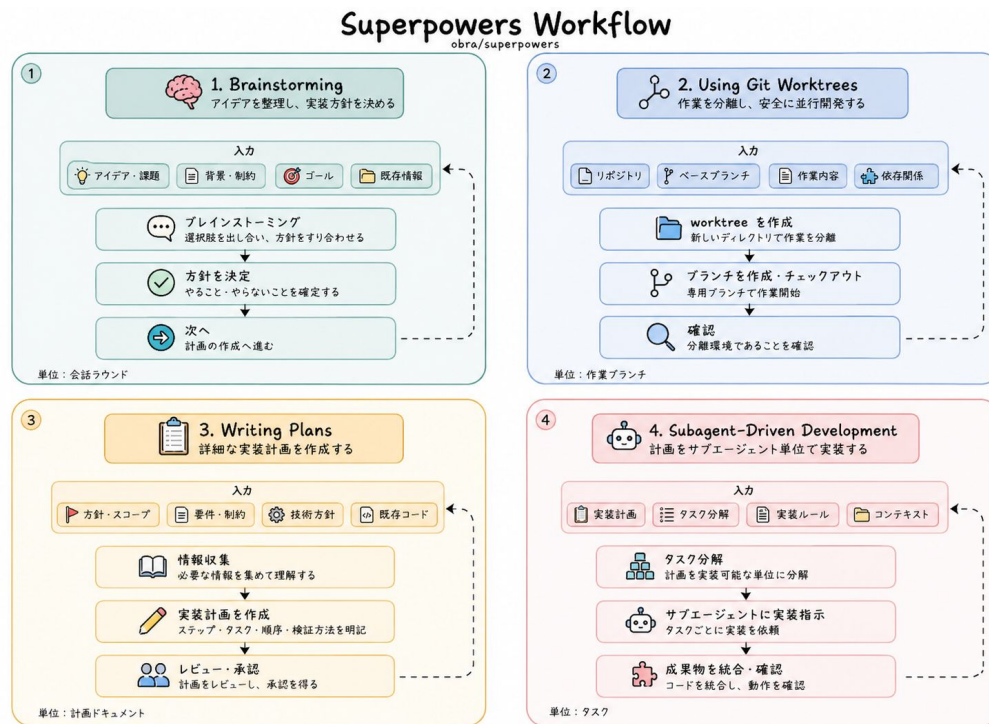


AI駆動開発のフレームワーク：AI-DLC

- AI-DLCには理論と実践が存在するが、理念を浸透させ、ワークフローを導入するのは期間的にも厳しかった。部分的な導入に関してもメリットを説明できず、見送り。
- AI-DLCは開発プロセス全体に大幅な見直しを強いるものであり、単にルールを導入するだけではなく、既存プロセスとの対応や会社標準との適合性の検討が必要であり、実現が難しかった。
- 一言でいうと「重たかった」

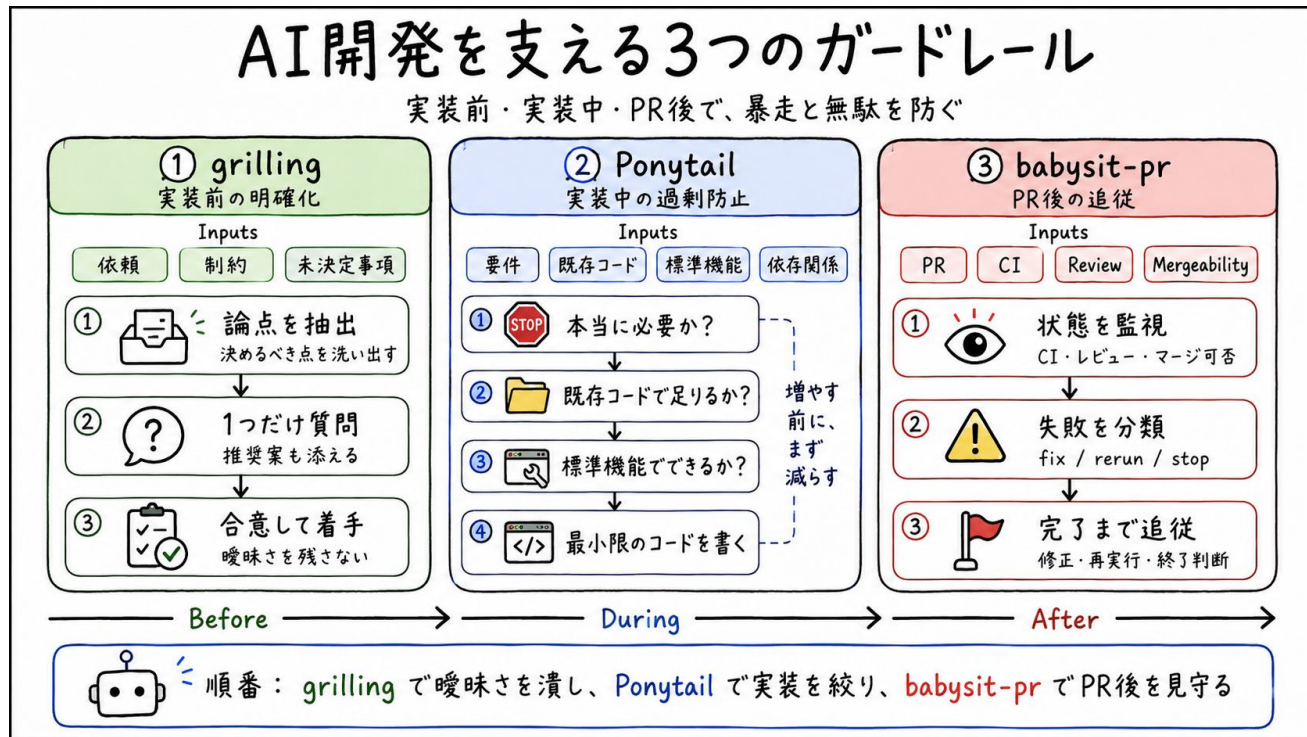
AI駆動開発のフレームワーク：ベスプラSkills系

例1：obra/superpowers(※抜粋・AI作図)



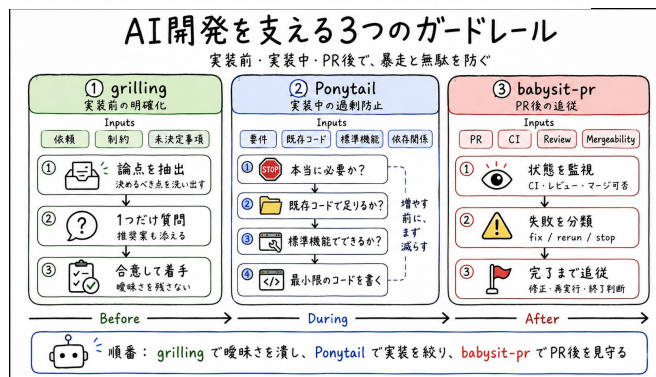
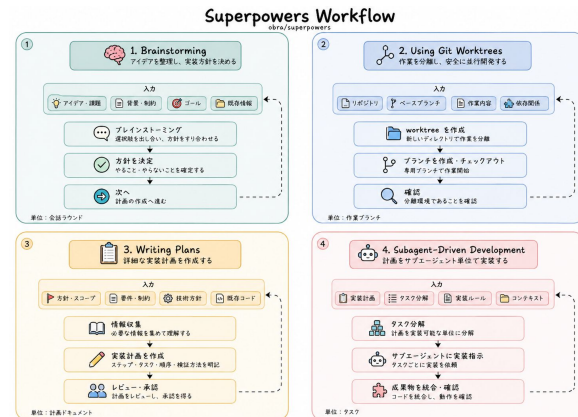
AI駆動開発のフレームワーク：ベスプラSkills系

例2：Skillsの組み合わせ(※AI作図)



AI駆動開発のフレームワーク：ベスプラSkills系

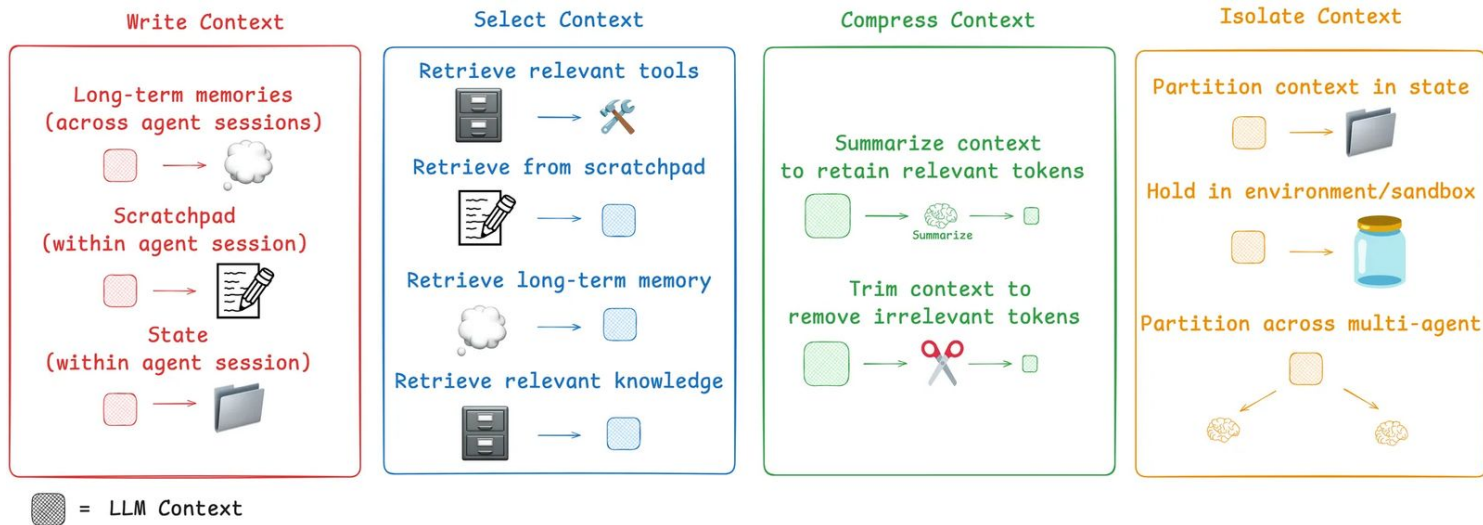
- 個別の推奨プロセスとしては良いが、ワークフローの型としては不足するように感じた
- 標準化目的のハーネスには何が足りていないか？
 - 人間のための コンテキストエンジニアリング



人間のための コンテキストエンジニアリング

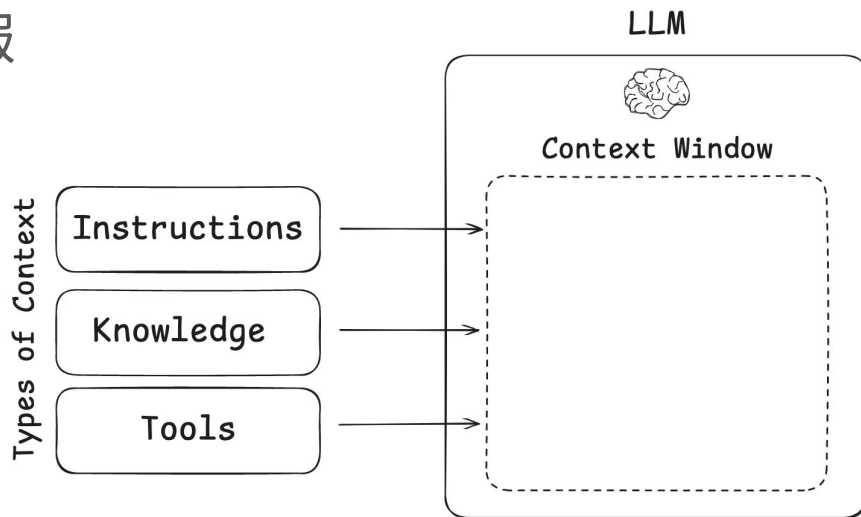
Context Engineering

コンテキストを書く / 選ぶ / 縮める / 分ける ための技術



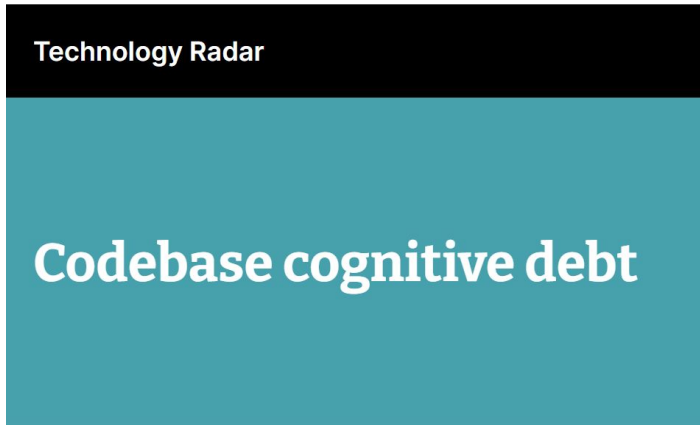
LLMが参照できる範囲

- Context : プロンプト + 外部情報
- Context Window : LLMの理解できるトークンの総容量
- つまり、LLMとの「会話」において、総トークン数がContext Window以内である必要がある



人間が参照できる範囲

- Thoughtworks Technology Radarが「codebase cognitive debt(認知負債)」を注意対象として採用
- > (私訳)コードベースの認知的負債とは、システムの実装と、「そのシステムがどのように、なぜそのように動作するのか」というチーム内の共通理解との間に生じる隔たりが、徐々に拡大していく状態を指します。

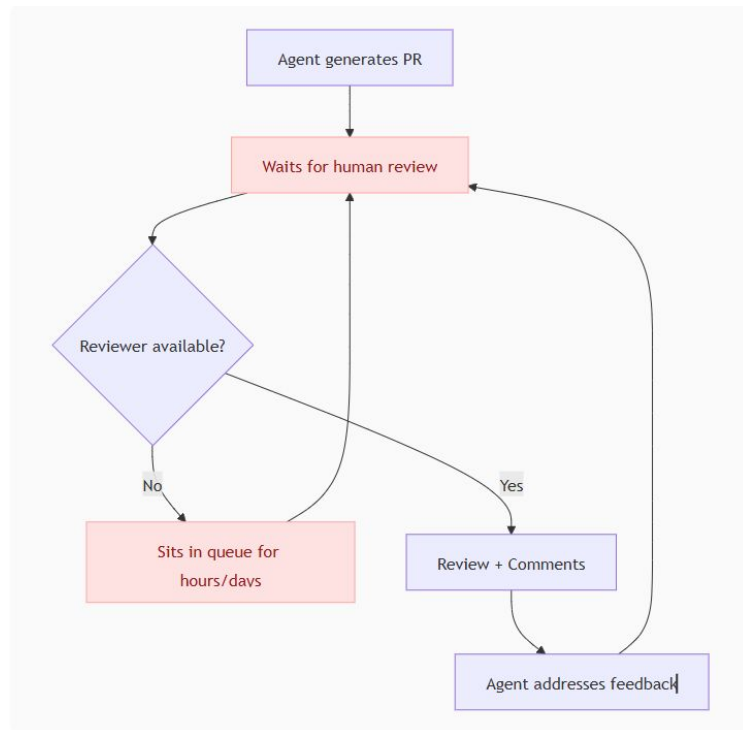


”理解負債”について

- 2026年の研究では、207人の学生が8週間にわたり記録した日誌から、GenAIによる「Comprehension Debt」が分析された。
 1. ブラックボックスとしてコードを受け入れる
 2. プロジェクト文脈と合わないコードを採用する
 3. AI依存により自力で扱う能力が低下する
 4. 検証を省略する

”認知負荷”は開発のブレーキになる

- 人間の理解する速度はAgentにとってのブレーキになり、開発が滞留する
 - Agentの速度は500PR/day
 - チームのレビュー能力は10PR/day
- ”儀式”の見直し案
 - レビューをコード生成の一部とする
 - Agentが自身の作業を検証する
 - 敵対的なAgentにレビューさせる



認知(理解)負債と認知負荷

- エージェントの生成物を理解できていないと負債になる
- エージェントの生成速度で理解すると開発速度が遅れ、レビューに負荷がかかる
- 理解するためのフォーマット＝形式、及びそれを扱うワークフローが欲しかった

辿り着いたのは仕様駆動開発だった

欲しかった「守」

- 「AI駆動開発の進め方」として欲しかったもの
 - 開発のための規律
 - 理解のための形式
 - レビューのためのワークフロー、生成物
- 「破りやすい」型が欲しかった
 - AI-DLCは重厚長大な「型」なので導入後の sunk コストやロックインの影響が気になった
 - Skillsの寄せ集めの「型なし」では不安があった

仕様駆動開発の選定

- KiroのSpec機能はスコープから外した
 - エージェント選定の自由を考慮
 - モデル性能・トークンコストなど考慮が必要
- Spec-Kit vs OpenSpec
 - Spec-Kitの生成するドキュメントは重厚長大になりやすく、認知負荷が高すぎると判断し、OpenSpecを選択した。
- OpenSpecの良いコマンド
 - `/opsx:verify` : 仕様と実装の一次チェック
 - `/opsx:archive` : 実装完了後のSpecをarchiveし、メインの仕様ファイルへ同期する

自然言語の”仕様”に限界はある



mizchi ✓
@mizchi



↻ Show translation

機械的に検査できない自然言語をマスターとして接地させようとする限り仕様駆動は失敗する。

LLM を形式手法とドメインの翻訳機として使うべきで、人間側は自然言語で問い合わせるが、自然言語そのものを仕様として使うべきではない。

5:55 PM · Jul 16, 2026 · **59.7K** Views

プロジェクトのSpecファイルを剥がす例

 **Takuto Wada**
@t_wada

⊗ ...

 [Show translation](#)

詳細な設計書からコードはいつでも再生成という実験は上手いかず（文書間の整合性を保つのはコードの整合性を保つより難しい）、文書が不要なくらい綺麗なコードを書き（書かせ）、そのうえでコードに書けないこと（Why, Why not）を文書にしっかり残し、全てをバージョン管理するスタイルに戻した

 **Takuto Wada** @t_wada · Jul 6

自分の開発スタイルと合わなくなってきてしまったので SpecKit 剥がしを行っている

このセッションでは、このプロジェクトから SpecKit を割いたのが GitHub Flow + ADR スタイルへのドキュメント化だ。ADR は仕様と経験を管理するものが散らばってしまっている状態を、ひとりの下に SpecKit で作成した仕様が揃っている状態で、早く右脳が活用できる。このプロジェクトを GitHub Flow + ADR の書籍にしようとしていると考えています。途中から戻しかけているのが docflow という形になっていますが、本来はこれに 001 であるべきではない。SpecKit で開発した内容と decision が決まっているからです。

のセッションでは、あたらしく最初のSpecKitを使わずにGitHub/FLOWなようなためにドキュメントを再整理して見なさい。Specはディレクトリ以下で問題を解する。現在の設計はdocs/design.txtに1時の意図決定とdocs/adr以下に、特定の例を導くように直して記述していき、そのスタイルに再整理してください。まずは再整理のための計画を立て、私の承認を得てください。

→ docs/adr/ に時系列で番号を振り直し、追

履歴を解析して、docs/design + docs/adr への再整備計画を立てます。まず計画

ト、使用済み2個のツール・

Mode,ExitPlanMode

した

specs/、.specify/、.claude/commands/spect
と ADR に残る)【ユーザー確認済み】

×モ4件 → docs/notes/ に移動して保持【ユー

れない。ドキュメントと設定ファイルのみの型

仕様駆動開発の消費期限＝型を破るタイミング

- 「仕様駆動開発」の既知の課題
 - コーディングからレビューへ移行するボトルネック
 - 仕様ファイル群の整合性チェック(ドリフト管理)
 - 仕様という言葉に拘束され、LLMの性能を発揮できない例
 - 最新LLMのベストプラクティスは「プロンプトを減らす」
- 逆にいうと、課題に突き当たるまでは仕様駆動開発は理解しやすい
 - メンバの作業を適宜見ながら意識合わせするという、チーム仕事の基本
- チーム全員が消費期限切れと感じたら、仕様駆動開発の次へ進めば良い